

STAGE S0

Problem Definition & Need Identification

Consolidated Evidence — Steps 1 through 7, in working order

Team No.	1
Theme No.	1
Artefact Name:	Mini RDBMS Engine with Query Processing & Transaction Support
Team Members	1.Bhoomi Bankapur 2.Sneha Shettar 3.Shubham Mishra 4.Soumya Patil
Guide Name	Ms. Jyothi Patil Mam

Order of Work — What Gets Done First

This pack is sequenced in the order the work actually happens: understand the problem and its people first, map the current process, translate observations into needs and objectives, then bound the scope and check it's achievable.

- 1.** The Problem — current situation, observed issue, why existing approach is inadequate
- 2.** Who Is Involved — stakeholder role, need, difficulty faced, expected benefit
- 3.** How It Works Today — current process before the proposed system: actors, steps, pain points
- 4.** Problem & Goals — one concise computing problem, bounded and evidence-supported, plus the objectives it drives
- 5.** What's Included — in-scope and out-of-scope items; avoid unrealistic feature expansion
- 6.** Can We Do This — technical/operational feasibility, constraints, and initial risks
- 7.** What Could Go Wrong — project-level risks and mitigation, revisited at later gates

THE PROBLEM

Step 1 of 7 — S0 Problem Definition & Need Identification

Stage S0 — Problem Definition & Need Identification | General, Theme-Independent

1. Foundational Courses

Sl.No.	Course	Core Concepts
1.	DBMS	Relational model, tables/tuples/attributes, primary/foreign keys, constraints, ACID properties, normalization
2.	OSPP	Process/thread synchronization, locking (mutex/semaphore), deadlock detection & avoidance, file I/O, memory management
3.	DAA	Searching & sorting algorithms, time complexity (Big-O) analysis, indexing structures (hash tables/B-trees)
4.	POCD	Lexical analysis, grammar (BNF/CFG), parsing techniques, syntax tree construction, semantic analysis

2. Existing Problem / Current Situation

Describe the real-world problem or gap as it exists today. What currently happens, and where does it fall short?

Current Situation

- Traditional database systems are large and complex for learning and experimentation.
- Query processing and transaction handling are usually hidden inside the DBMS.

Observed Issue

- Students cannot easily understand how queries are processed internally.
- It is difficult to visualize indexing, query execution, storage management, and transactions together.

3. Who Is Affected

Briefly name the affected users/groups. (Full stakeholder detail belongs in the separate Stakeholder Analysis document.)

1. Students / Users

2. Faculty

3. Developers / Team Members

4. System Administrator

4. Why the Existing Approach Is Inadequate

Explain specifically what is missing, slow, error-prone, manual, or unreliable about the current approach.

- Existing DBMSs provide the final result but hide the internal working.
- They do not provide a simple platform to demonstrate DBMS, DAA and OSPP concepts together.
- A lightweight system is needed to understand and demonstrate these concepts practically.

5. Evidence That This Problem Is Real

List observations, informal surveys, interviews, existing data, or references that support the problem's existence — not assumptions.

- DBMS, OS, and DAA concepts are taught separately.
- Students rarely integrate these concepts into one working system.
- Most students use databases through high-level SQL tools or ORMs.
- Internal transaction mechanisms remain largely a **black box**.
- Concepts like **ACID, concurrency, locking, and recovery** remain theoretical.
- Real DBMSs are too complex to study at the source-code level.
- A simplified DB engine can bridge the **theory-implementation gap**.
- Hands-on implementation helps students understand DBMS internals deeply.

6. Sources / References

Cite where your evidence came from (people consulted, documents reviewed, existing systems observed, etc.).

KLE Tech University DBMS, OSPP, DAA, and POCD course curricula
Existing open-source RDBMS documentation reviewed for architectural reference
University project precedent: similar mini-DBMS-engine assignments referenced from public course repositories (e.g., CMU 15-445 Database Systems, UC Berkeley CS186)

 **REMINDER: This document must be dated and completed BEFORE any requirements engineering (S1) work begins — Gate 0 checks this explicitly.**

WHO IS INVOLVED
Step 2 of 7 — S0 Problem Definition & Need Identification
Stage S0 — Problem Definition & Need Identification | General, Theme-Independent

1. Stakeholder Table

List every distinct group or role that interacts with, is affected by, or has an interest in this problem/system. Add rows as needed.

Stakeholder	Role / Description	Needs & Expectations	Current Pain Points	Influence (H/M/L)
1. Students / Users	Execute queries and transactions	Simple database system for learning	Difficult to understand internal DBMS working	High
2. Faculty	Guide and evaluate the project	Easy way to demonstrate concepts	Existing DBMS hides internal processes	Medium
3. Developers / Team Members	Design and implement the system	Clear modules and algorithms	Integrating DBMS, DAA and OSPP concepts	High

Stakeholder	Role / Description	Needs & Expectations	Current Pain Points	Influence (H/M/L)
4. System Administrator	Manage data and system operations	Reliable storage and transactions	Managing data consistency	Medium

2. Primary vs. Secondary Stakeholders

Primary: directly use or are directly affected by the system. Secondary: indirectly involved (e.g., administrators, oversight bodies).

Primary Stakeholders	Secondary Stakeholders

3. Key Insight Summary

In 3–5 sentences, summarize what the stakeholder analysis reveals about whose needs the system must prioritize.

💡 REMINDER: Every stakeholder listed here should be traceable to at least one requirement in S1.

HOW IT WORKS TODAY

Step 3 of 7 — S0 Problem Definition & Need Identification

Stage S0 — Problem Definition & Need Identification | General, Theme-Independent

1. Current Process — Narrative Summary

Describe, in plain language, how the process/task is currently carried out from start to finish.

The Students/developers needing structured data storage today either:
a) install a full production RDBMS (e.g. MySQL, PostgreSQL) and use it as a black box or
b) roll ad-hoc storage using flat files, lists or dictionaries in code.
In both the cases, the internal mechanics like parsing, query execution, transaction handling will stay invisible. SQL is learned as a syntax to type not a process to understand.

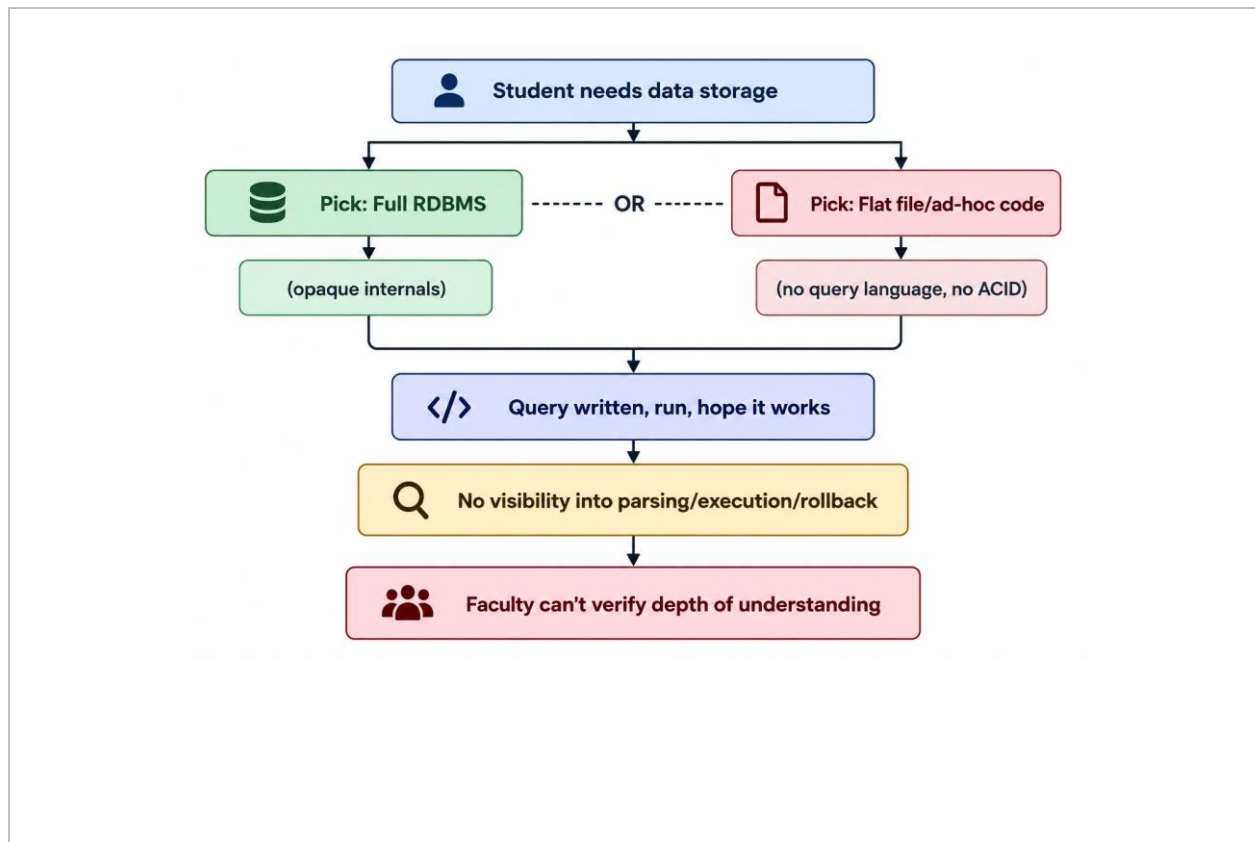
2. Step-by-Step Workflow Table

Break the current process into individual steps. Add or remove rows as needed.

Step No.	Actor / Role	Action Performed	Tool / System Used	Pain Point Observed
1.	Student/developer	Chooses a storage solution.	MYSQL or flat files	Either too complex to inspect, or no query/transaction support at all.
2.	Student/developer	Writes and runs queries	RDBMS client/app-code	No visibility into parsing or execution plan
3.	Student/developer	Performs multi-step operations	App-level code	No atomicity so a mid-write leaves corrupted data
4.	Student/developer	Tries to learn DB internals	Textbooks or lecture diagrams	Theory is disconnected from a hands on and inspectable system.
5.	Faculty/evaluator	Assesses DB understanding	Theory exams	NO practical artifact to verify implementation level understanding.

3. Workflow Diagram

Attach or paste a flowchart / swim lane diagram of the AS-IS process here (insert image after printing/exporting, or paste into this space in the digital copy).



4. Bottlenecks & Inefficiencies Identified

Summarize the key weaknesses in the current workflow that the proposed system should address.

- Query processing is a black box — no exposure to parsing, planning, or execution internals
- No transaction/ACID guarantees in typical student-built storage — data integrity breaks on crash or concurrent access
- Production RDBMS installs bring configuration overhead that obscures the core concepts being taught
- Flat-file storage has no schema enforcement, no query language, no consistency guarantees

💡 REMINDER: This document proves you understand the existing process before proposing a new one — required as Gate 0 evidence.

PROBLEM & GOALS

Step 4 of 7 — S0 Problem Definition & Need Identification

Stage S0 — Problem Definition & Need Identification | General, Theme-Independent

1. Observed Problems → Underlying Needs

For every observed problem, state the evidence/source behind it, the underlying need it points to, and which stakeholder(s) it affects. Add rows as needed.

Observed Problem	Evidence / Source	Underlying Need	Linked Stakeholder(s)
RDBMS internals are a black box for students	CS curriculum teaches SQL usage, not engine internals	A transparent, buildable engine to learn from	faculty/mentor
Ad-hoc storage lacks query capability	Common in student projects without DB coursework	A lightweight query processor	You, apps built on top
No transaction safety in simple storage	Data corrupts on partial writes/crashes	ACID-like atomicity and rollback	End users/apps of the engine
Heavy commercial RDBMS obscure fundamentals	Complex setup for MySQL/PostgreSQL as a learning tool	A minimal, inspectable implementation	Evaluators
No artifact proves DB systems depth	Lab exams test usage, not implementation	A working system as evidence of mastery	Faculty/examiners

2. Prioritization

Rank the needs above by importance/urgency, and briefly justify the ranking.

- Transaction support — highest priority; hardest to get right, biggest differentiator in the demo (rollback "wow" moment)
- Query processing (parser + execution) — core functional requirement, directly named in the title
- Storage engine — foundational; everything else depends on it
- Interface layer — necessary for demo, lowest technical complexity

3. Problem Statement

State the single, concise computing problem this project solves — bounded, evidence-supported, and free of solution language.

Students and developers lack a transparent, minimal RDBMS implementation that demonstrates core query processing and transaction management, since production databases are too complex to inspect internally and ad-hoc storage solutions provide neither a query language nor transactional integrity.

4. Project Objectives

State 4–6 clear, measurable objectives this project aims to achieve, based on the needs above. Each should be specific enough to later map to a requirement or deliverable.

1. Design a storage engine supporting table/schema definition with typed columns and constraints (PRIMARY KEY, NOT NULL).
2. Implement a SQL-subset parser supporting CREATE, INSERT, SELECT (with WHERE), UPDATE, DELETE.
3. Build an execution engine that processes parsed queries against storage and returns correct results.
4. Implement transaction management with BEGIN/COMMIT/ROLLBACK and atomicity guarantees.
5. Demonstrate a rollback scenario where a failed transaction leaves zero partial changes.
6. Provide a CLI interface for issuing queries and viewing results/errors.

💡 **REMINDER: Every need and objective here should map forward into a Functional or Non-Functional Requirement in S1 — keep this page for your Requirements Traceability Matrix later.**

WHAT'S INCLUDED**Step 5 of 7 — S0 Problem Definition & Need Identification****Stage S0 — Problem Definition & Need Identification | General, Theme-Independent**

1. In-Scope

List what the system *WILL* do / include within this semester's project.

#	In-Scope Item
1	Core SQL-like RDBMS Engine Implement CREATE TABLE, INSERT, SELECT, UPDATE, DELETE, WHERE, and basic relational data storage.
2	Intelligent SQL Error Explanation Build a lexer, parser, semantic checker, and beginner-friendly error messages with column suggestions
3	Safe UPDATE / DELETE Preview Preview affected rows, show before/after values, and require confirmation for destructive operations.
4	Transaction Support and Recovery Implement BEGIN, COMMIT, ROLLBACK, undo logging, and a focused crash-recovery mechanism.
5	

2. Out-of-Scope

List what the system *explicitly will NOT* do / include — this prevents scope creep later.

#	Out-of-Scope Item	Reason for Exclusion
	Full SQL standard support	Too large for a 15-week student project.
	Distributed database and replication	Requires multiple servers, networking, and distributed consistency.
	Advanced concurrency with MVCC and multiple isolation levels	High implementation and testing complexity.
	Advanced query optimizer and hybrid row-column storage	Requires sophisticated cost models, storage design, and benchmarking.

#	Out-of-Scope Item	Reason for Exclusion
	Production-grade GUI, cloud deployment, and enterprise security	Not essential to demonstrate the core database engine

3. Assumptions

List anything assumed to be true for planning purposes (e.g., available data, user access, environment).

The system is assumed to run on a single computer using Python 3 and local file storage. Users are assumed to have basic SQL knowledge and access to the application. The project will use a limited, documented SQL-like grammar and a small to medium-sized dataset suitable for classroom testing. The team will have access to the required software, development tools, and sample datasets throughout the 15-week semester.

4. Constraints

List known limitations — time, team size, technology, data availability, etc.

The project is limited to a 15-week semester timeline and a student-sized development team. The engine must be implemented from scratch using available programming skills and open-source tools. The supported SQL grammar, storage format, and transaction model will be limited to maintain feasibility. Hardware resources are restricted to ordinary student laptops, and the system will not be designed for production-scale workloads, distributed execution, or enterprise-level reliability.

💡 REMINDER: A clear Out-of-Scope list is just as important as In-Scope — reviewers will check both at Gate 0.

CAN WE DO THIS

Step 6 of 7 — S0 Problem Definition & Need Identification

Stage S0 — Problem Definition & Need Identification | General, Theme-Independent

1. Technical Feasibility

Assess whether the team has (or can acquire) the technical skills, tools, and technology needed.

Aspect	Assessment	Justification
Programming	Feasible	Python supports file handling, data structures, and modular engine development
Compiler Design	Feasible with learning	Lexer, grammar, recursive descent parsing, and AST construction can be implemented for a limited language.
DBMS & DAA	Feasible with learning	Relational algebra, indexing, query execution, and basic transaction algorithms are suitable for student implementation.
OS concepts	Partially feasible	File I/O, locking, buffering, and recovery can be implemented at a basic level without developing a full operating system.

2. Operational Feasibility

Assess whether the proposed system will realistically work within the target environment/users.

The proposed system can operate on a normal student laptop through a command-line interface or a simple application interface. Users can create tables, insert records, execute supported SQL-like queries, preview destructive modifications, and manage transactions. The system is designed for educational use and small-scale testing. Its limited SQL grammar and single-node execution model make it easier to operate, understand, and debug than a full-scale database system.

3. Time Feasibility

Assess whether the scope is achievable within the semester timeline (15 weeks).

Go — The Mini RDBMS Engine with Query Processing & Transaction Support is technically, operationally, financially, and time feasible within the 15-week semester. The project uses accessible programming tools and established algorithms such as lexical analysis, parsing, semantic checking, relational algebra, undo logging, and basic transaction management. The scope is limited to a single-node SQL-like engine with core query processing, beginner-friendly error explanations, safe modification previews, transactions, and lightweight execution tracing. Advanced database features are excluded to ensure the team can implement, test, and demonstrate a working system within the available resources and timeline.

4. Resource Feasibility

Assess team size, skill distribution, and access to any needed data/hardware/software.

The project requires a student development team with basic knowledge of Python, data structures, DBMS, compiler design, and operating system concepts. The implementation can be developed using ordinary laptops, Python 3, VS Code, Git, and local file storage. No specialized hardware, paid database services, or cloud infrastructure are required. Team members can divide the work into parser and error handling, query execution and storage, transaction management and recovery, and testing and documentation

5. Overall Feasibility Conclusion

State a clear go / go-with-modifications / no-go conclusion, with reasoning.

Go — The Mini RDBMS Engine with Query Processing & Transaction Support is technically, operationally, financially, and time feasible within the 15-week semester. The project uses accessible programming tools and established algorithms such as lexical analysis, parsing, semantic checking, relational algebra, undo logging, and basic transaction management. The scope is limited to a single-node SQL-like engine with core query processing, beginner-friendly error explanations, safe modification previews, transactions, and lightweight execution tracing. Advanced database features are excluded to ensure the team can implement, test, and demonstrate a working system within the available resources and timeline.

💡 REMINDER: Feasibility must be realistic, not optimistic — reviewers will check this against your stated scope and timeline.

WHAT COULD GO WRONG

Step 7 of 7 — S0 Problem Definition & Need Identification

Stage S0 — Problem Definition & Need Identification | General, Theme-Independent

1. Project-Level Risks

Identify risks that could affect the project at this early stage — technical, resource, scope, or timeline related. Add rows as needed.

Risk ID	Risk Description	Category	Likelihood (H/M/L)	Impact (H/M/L)	Mitigation Strategy
1	Rollback leaves inconsistent data	Technical	Medium	High	Early, repeated rollback testing
2	Deadlocks/race conditions in concurrency	Technical	Medium	High	Simple locking mechanism, test under simulated concurrent access
3	Timeline too tight for full ACID support	Timeline	High	High	Build MVP first, treat extras as stretch goals
4	Modules fail to integrate cleanly	Integration	Medium	High	Define module interfaces before coding separately
5	Uneven team skill on modules	Resource	Medium	Medium	Pair members, document code for handoff
6	Partial commits leave half-applied changes	Technical	Medium	High	Use write-ahead log; apply changes only on commit

2. Risk Owner Summary

For each risk above, note which team member is responsible for monitoring it (optional but recommended).

Risk	Owner	Monitoring
Inconsistent state from rollback	Bhoomi	Weekly
Deadlock with no detection/timeout	Sneha	Weekly
Modules don't integrate smoothly	Shubham	At each milestone
Disk I/O errors unhandled	Soumya	Weekly
Uneven team skill on modules	Bhoomi	Weekly
Partial commits leave half-applied changes	Sneha	Weekly

 **REMINDER:** This is an INITIAL register — it will be revisited and expanded at later gates as the project matures.

